

ANDROID

UNITY ECS

Addressable

# SLIME DEFENSE

Unity ECS/DOTS 기반 모바일 타워 고정형 성장 디펜스 게임

Unity 개발자 포트폴리오 · 1인 개발 · Android 빌드

제작자 : 안병찬

Tel : 010-5408-9821

E-Mail : ahnhea0107@naver.com

Git : [https://github.com/eggmoney1073/Slime\\_Defense](https://github.com/eggmoney1073/Slime_Defense)

설명 및 요약 영상 : [https://www.youtube.com/watch?v=3Ujpl2\\_W3qQ&t=20s](https://www.youtube.com/watch?v=3Ujpl2_W3qQ&t=20s)

게임 풀 영상 : [https://www.youtube.com/watch?v=FuFSvaY\\_o\\_8&t=170s](https://www.youtube.com/watch?v=FuFSvaY_o_8&t=170s)

2026.03.22 ~ 2026.05.07



Wave Spawn

Level Up

# 목차

01

OVERVIEW

## 프로젝트 개요

플랫폼 · 장르 · 개발 기간  
게임 콘셉트와 플레이 루프

02

ARCHITECTURE

## 구조 설계

프로젝트 구조 · ECS 흐름  
Entity / Buffer 설계

03

CODE

## 예시 코드

Grid 충돌 후보 검사  
Damage Buffer · Addressables

04

PROBLEM

## 문제 해결

빌드에 SubScene 누락 문제  
투사체 충돌 누락 원인 분석

05

RESULT

## 결과와 확장 방향

검증 결과 · 강조 역량  
확장 방향 · 프로젝트 링크

# 프로젝트 개요

## 포트폴리오 핵심

"Unity ECS/DOTS를 실제 모바일 게임 구조에 적용하고, Android 환경에서 Addressable 씬 로딩과 대량 Entity 처리·충돌 판정을 검증한 1인 개발 프로젝트입니다."

플랫폼

Android

실기기 APK 실행 확인

장르

디펜스 + 서바이벌 성장형

중앙 고정형 자동 전투

개발 형태

1인 개발

기획·프로그래밍·빌드  
검증 담당

개발 기간

2026.03.22 ~ 2026.05.07

README 정리

핵심 목표

Unity ECS/DOTS 적용

대량 Entity 처리 구조  
검증

# 게임 콘셉트와 플레이 흐름

## 핵심 콘셉트

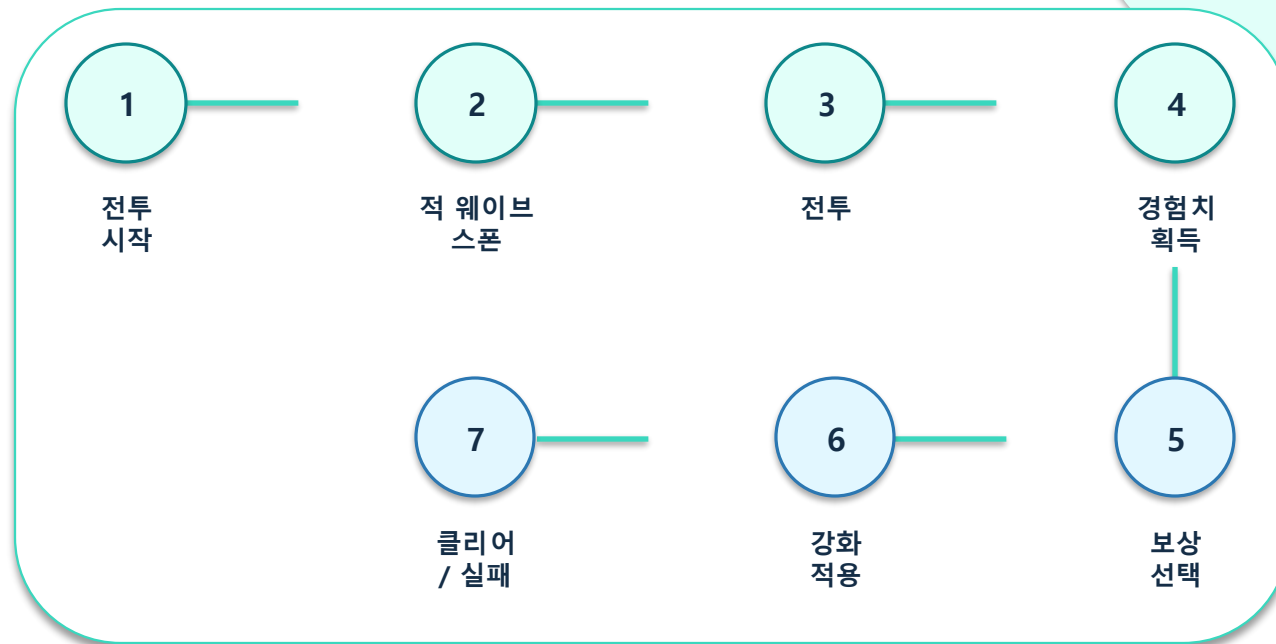
중앙에 고정된 플레이어가 몰려오는 슬라임을 전투로 막는 모바일 디펜스 게임입니다.  
전투 중 경험치를 획득하고 레벨업 보상으로 무기와 능력치를 강화합니다.

## 승패 조건

제한 시간 동안 생존하면 클리어, 플레이어 체력이 소진되면 실패합니다.

## 설계 포인트

플레이 루프를 먼저 정의한 뒤 Spawn → Move → Judgement → Calculate → Destroy 순서의 ECS SystemGroup으로 분해했습니다.



# 프로젝트 구조

## Folder Structure

```
Assets/
├── 1.Scenes/           Bootstrap · Loading · Lobby · Game · SubScene
├── 2.Scripts/
│   ├── ECS/
│   │   ├── Authorings  MonoBehaviour → Entity 변환
│   │   ├── Bakers      Baker 구현
│   │   ├── Bridge       ECS ↔ Mono 연결
│   │   ├── Entity Structs  Component / Buffer 정의
│   │   └── Entity Systems  Spawn · Move · Judgement · Calculate · Destroy
│   ├── Loading          Addressables 씬 로딩
│   ├── Download          에셋 다운로드 관리
│   ├── Input / UI        플레이 입력과 화면 표시
│   ├── Utility            Singleton
│   └── AddressableAssetsData  Addressables 설정
```

## 구조 설계 의도

전투 로직은 ECS에 집중시키고, 화면 표시·로딩·입력 같은 관리 영역은 MonoBehaviour 기반으로 분리했습니다.

## 관리 기준

Authoring/Baker는 변환 책임, Entity Structs는 데이터 정의, Entity Systems는 실행 책임으로 나누어 수정 범위를 좁혔습니다.

## UI 기능 분리

UI 개발이 게임 씬과 결합되지 않도록 Prefab으로 분리하여 개발했습니다.

# ECS SystemGroup 실행 흐름



## JudgementSystemGroup, CalculateSystemGroup 설계 판단

충돌 판정과 데미지 계산을 한 시스템에서 처리하지 않고, JudgementSystemGroup과 CalculateSystemGroup으로 나누었습니다. 이 구조 덕분에 병렬 처리와 데이터 반영 순서를 더 명확하게 관리할 수 있었습니다.

## DestroySystemGroup 설계 판단

사망 처리된 엔티티를 바로 삭제하는게 아니라 LiveTag를 비활성화 한 뒤 마지막 DestroySystemGroup에서 일괄 삭제하는 구조로 설계하여 병렬 처리 중에 엔티티가 삭제되는 현상을 막을 수 있었습니다.

# Entity / Component 설계

## Enemy Entity

EnemyTag

LiveTag

EnemyHealth

LocalTransform

ECS\_PathReference

Damaged Buffer

## Projectile Entity

ProjectileTag

MoveSpeed

LifeTime

PierceCount

HitEnemyBuffer

## Event Entity

GameClearEvent

LevelUpEvent

SoundRequest

WeaponRequest

### LiveTag 설계

죽은 상태를 DeadTag로 켜는 방식보다, 살아 있는 동안 LiveTag를 유지하고 제거 대상이 되면 비활성화하는 방식이 시스템 흐름에 더 직관적이었습니다.

### Buffer 설계

투사체 충돌 결과를 즉시 체력에 반영하지 않고 Damaged Buffer에 누적한 뒤 계산 단계에서 일괄 처리했습니다.

## 주요 구현 내용

### ECS 기반 전투 처리

적, 투사체, 충돌, 생존 처리를 Entity/Component/System 구조로 구현했습니다.

### Grid 기반 충돌 최적화

투사체가 전체 적을 검사하지 않고 주변 Cell만 검사하도록 후보군을 줄였습니다.

### Addressables 씬 로딩

Loading Scene을 유지하고 Lobby/Game Scene을 Additive로 교체하는 구조를 사용했습니다.

### 레벨업 선택 시스템

전투 중 경험치를 획득하면 3개 보상 후보 중 하나를 선택해 무기와 능력치를 강화합니다.

### ECS ↔ Mono Bridge

UI, 사운드, 레벨업 요청은 이벤트 Entity/Buffer를 통해 느슨하게 연결했습니다.

### Android 검증

APK 빌드 후 ADB 설치와 실기기 실행으로 Editor와 빌드 환경 차이를 확인했습니다.



# Grid 기반 충돌 후보 검사

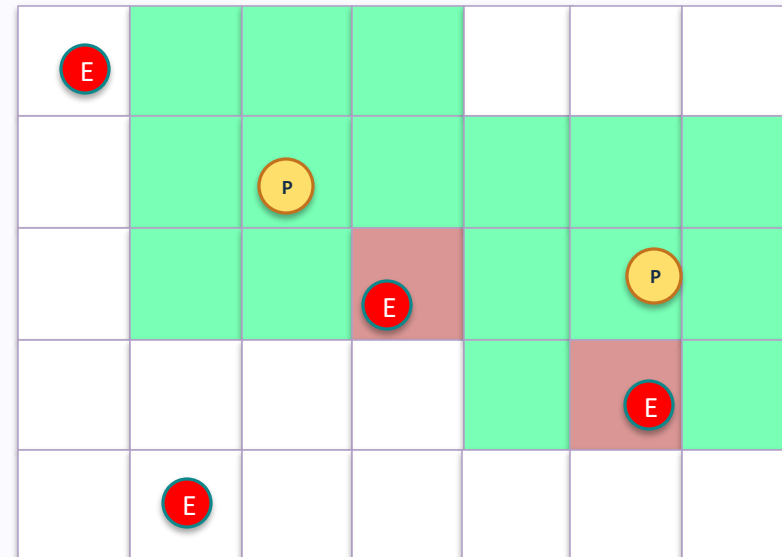
Collision\_System.cs / Collision\_Job.cs

```
NativeParallelMultiHashMap<int, Entity> _gridMap;

public void OnCreate(ref SystemState state)
{
    int capacity = 1024;
    _gridMap = new NativeParallelMultiHashMap<int, Entity>(capacity, Allocator.Persistent);
}

int centerX = (int)math.floor(position.x / _cellSize) + (_gridWidth / 2);
int centerY = (int)math.floor(position.y / _cellSize) + (_gridWidth / 2);

// 주변 3x3 Cell만 충돌 후보로 검사
int offsetY = -1;
while (offsetY <= 1)
{
    int offsetX = -1;
    while (offsetX <= 1)
    {
        int index = (centerX + offsetX) + _gridWidth * (centerY + offsetY);
        bool found = _gridMap.TryGetFirstValue(index, out other, out iterator);
        offsetX++;
    }
    offsetY++;
}
```



전체 탐색 대신 주변 3x3 Cell만 검사

## 효과

적과 투사체가 많아지는 상황에서 불필요한 전체 탐색을 줄였습니다.

# 데미지 Buffer / Addressables 로딩

## EnemyDamaged\_Job.cs

```
// Damage 반영: 충돌 결과를 Buffer에 누적 후 계산 단계에서 처리
public void Execute(ref DynamicBuffer<Damaged> damage, ref EnemyHealth health)
{
    if (damage.Length == 0) return;

    for (int i = 0; i < damage.Length; i++)
    {
        health.health -= damage[i].damage;
    }

    damage.Clear();
}
```

## LoadingSystem.cs

```
// Addressables Scene Load
string sceneAddress = _sceneBaseAddress + sceneName.ToString() + ".unity";
_contentSceneHandle = Addressables.LoadSceneAsync(
    sceneAddress,
    LoadSceneMode.Additive
);
```

## Damage Buffer

충돌 Job에서 체력을 직접 변경하지 않고 Damaged Buffer에 기록한 뒤 Calculate 단계에서 Buffer를 읽어서 체력에 반영했습니다.

## 장점

적의 컴포넌트를 병렬 처리 과정에서 동시에 접근하는 문제가 생기는 것을 방지할 수 있습니다.

## Scene Loading

Loading Scene을 유지한 상태에서 Lobby/Game Scene을 Addressables Additive 방식으로 교체했습니다.

## 장점

로딩씬 로드가 딜레이 없이 빠르게 반응하여 유저의 경험을 해치지 않게 했습니다.

## ECS SubScene<sup>0</sup>이 APK에 포함되지 않는 문제

### Before

Addressables Game Scene

SubScene

Editor에서는 정상 동작했지만 Android APK 빌드 후 ECS 데이터가 로드되지 않았습니다.

게임 씬을 Addressables로 로드하면서 Entity Scene 포함 처리가 누락되었습니다.

### After

Bootstrap

SubScene

APK

항상 포함되는 Bootstrap Scene에서 SubScene을 직접 참조하도록 변경했습니다.

그 결과 Android 실행 환경에서도 ECS 데이터가 정상 로드되었습니다.

### 배운 점

Addressables Scene 로딩 완료와 ECS Entity Scene 준비 완료는 같은 의미가 아니라는 것을 알게 되었습니다.

SubScene의 빌드 포함 관계와 로드 타이밍을 별도로 검증해야 한다는 것을 알게 되었습니다.

# 투사체 충돌 누락

## 문제 상황

적과 투사체가 많아질수록 일부 충돌이 누락되었습니다.

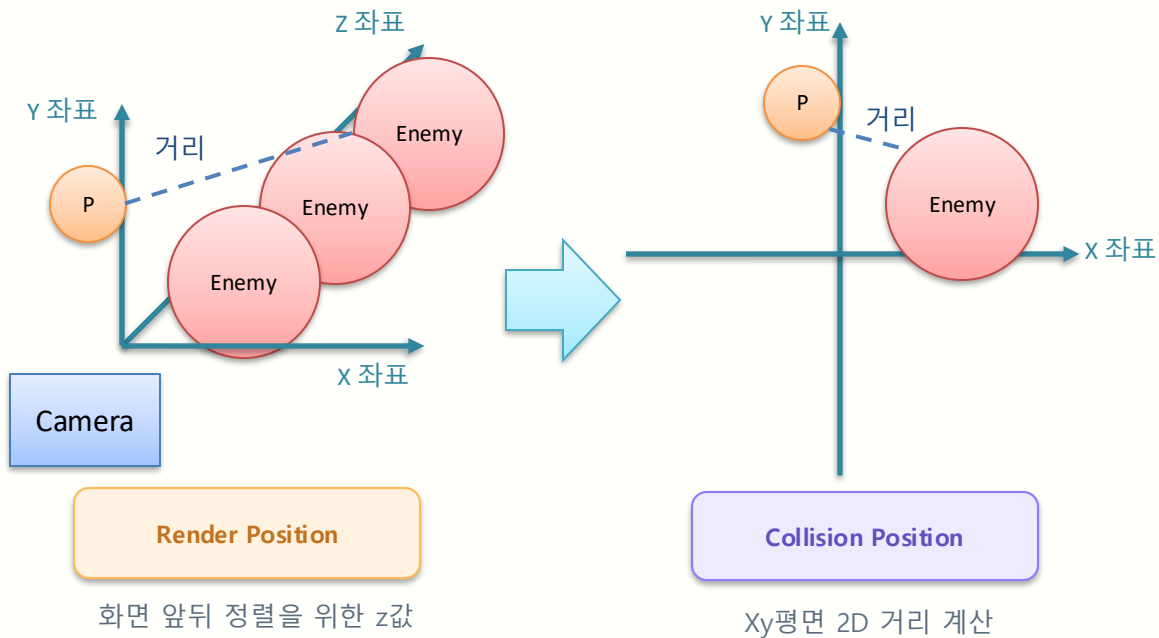
## 원인 분석

처음에는 Grid Buffer 용량 부족을 의심했지만, 실제로는 렌더링 정렬용 좌표 수정이 문제였습니다.

## 해결

렌더링 정렬을 위한 좌표인 z 좌표를 고려하지 않고, xy좌표의 거리를 기반으로 충돌 거리 확인을 했습니다.

## 좌표 기준 분리



렌더링 안정성 + 충돌 판정 안정성 분리 확보

## 프로젝트 결과와 확장 방향

# THANK YOU

## Slime Defense

### 검증한 것

ECS 기반 대량 Entity 처리  
Grid 기반 충돌 최적화  
Addressables + ECS SubScene 로딩  
Android APK 빌드와 실행

### 강조할 역량

구조 설계  
문제 원인 분석  
빌드 환경 검증  
성능을 고려한 데이터 흐름 설계

### 추가 개선 방향

밸런스 데이터 분리  
Addressables 패치 구조 확장  
스테이지/무기 콘텐츠 확장

### 마무리 메시지

개발 흐름을 끝까지 완성한 경험을 통해 한걸음 더 발전할 수 있었습니다.

제작자 : 안병찬

Tel : 010-5408-9821

E-Mail : ahnhea0107@naver.com

Git : [https://github.com/eggmoney1073/Slime\\_Defense](https://github.com/eggmoney1073/Slime_Defense)

설명 및 요약 영상 : [https://www.youtube.com/watch?v=3Ujpl2\\_W3qQ&t=20s](https://www.youtube.com/watch?v=3Ujpl2_W3qQ&t=20s)

게임 풀 영상 : [https://www.youtube.com/watch?v=FuFSvaY\\_o\\_8&t=170s](https://www.youtube.com/watch?v=FuFSvaY_o_8&t=170s)